Computer Science Department
California State University, Fullerton

CPSC 240 Computer Organization and Assembly Language
Final-Exam Example
Tuesday, December 16, 2025

Student Name: _____

Last 4 digits of ID: _____

Note:

- University regulations on academic honesty will be strictly enforced.
- You have 75 minutes to complete this Exam.
- You can use calculator for the Exam.
- Close books, slides, and turn off the computer.
- Turn off or turn vibration your cell phone.
- Any content submitted after the due date will be regarded as a make-up exam.

1. What will be the **ah, al, sil** and **r10** registers after execution? What is in **lst, sum,** and **ave** variables before execution and after execution until the label "_stop"? Show answer in hex, full register size. ***Note***, pay close attention to the register sizes (8-bit, 16-bit, 32-bit, or 64-bit).

```
section .data
lst     db      2, 7, 4
len     dq      3

section .bss
sum     resb    1
ave     resb    1

section .text
global _start
_start:
        mov     rdi, lst
        mov     rsi, qword[len]
        mov     rdx, sum
        mov     rcx, ave
        call    calculate
_stop:
        mov     rax, 60
        mov     rdi, 0
        syscall

global calculate
calculate:
        mov     al, 0
        mov     r10, rsi
next:
        add     al, byte[rdi+r10-1]
        dec     r10
        cmp     r10, 0
        jne     next
        mov     byte[rdx], al
        cbw
        idiv    sil
        mov     byte[rcx], al
        ret
```

(24 points)

Memory	Offset	Value (Hex)	
		before(initial)	after
ave	+0	0x00	
sum	+0	0x00	
lst	+2		
lst	+1		
lst	+0		

Register	Value (Hex)
	after execution
ah	
al	
sil	
r10	

2. What will be in the **cl** registers after execution? What is in **n** and **sum** arrays before execution and after execution until the label "_stop"? Show answer in hex, full register size. ***Note***, pay close attention to the register sizes (8-bit, 16-bit, 32-bit, or 64-bit).

```
%macro compute 2
       mov    al, 0
       mov    cl, 1
%%next:
       add    al, cl
       inc    cl
       cmp    cl, byte[%1]
       jbe    %%next
       mov    byte[%2], al
%endmacro

section .data
n      db     2, 3, 4

section .bss
sum    resb   3

section .text
global _start
_start:
       mov    r10, 0
doLoop:
       lea    rsi, byte[n + r10]
       lea    rdi, byte[sum + r10]
       compute rsi, rdi
       inc    r10
       cmp    r10, 3
       jl     doLoop
_stop:
       mov    rax, 60
       mov    rdi, 0
       syscall
```

(26 points)

Memory	Offset	Value (Hex)	
		before(initial)	after
sum	+2		
sum	+1		
sum	+0		
n	+2		
n	+1		
n	+0		

Register	Value (Hex)
	After execution
cl	

3. What will be in the **al** and **bl** registers after execution. What is in **num1** and **num2** arrays before execution and after execution until the label "_stop"? Show memory and register answer in decimal or hexadecimal. *Note*, pay close attention to the register sizes for hexadecimal number (8-bit, 16-bit, 32-bit, or 64-bit).

```
%macro mac2   2
       mov    al, byte[%1]
       mov    bl, byte[%2]
       mov    byte[%1], bl
       mov    byte[%2], al
%endmacro

section .data
num1   db     0x17, 0x39, 0x6A
num2   db     0x5F, 0x40, 0x25

section .text
global _start
_start:
       mov    r10, 0
next:
       lea    rdi, byte[num1 + r10]
       lea    rsi, byte[num2 + r10]
       mac2   rdi, rsi
       inc    r10
       cmp    r10, 3
       jb     next
_stop:
       mov    rax, 60
       mov    rdi, 0
       syscall
```

(28 points)

Memory	Offset	Value (Hex)	
		before (initial)	after
num2	+2		
num2	+1		
num2	+0		
num1	+2		
num1	+1		
num1	+0		

Register	Value (Hex)
	After execution
al	
bl	

4. What will be in the **rsi** register after execution? What is in **lst** array before execution and after execution until the label "_stop"? Show answer in hex, full register size. *Note*, pay close attention to the register sizes (8-bit, 16-bit, 32-bit, or 64-bit).

```
section .data
lst     db      '2', '7', '5', '3', '8'
len     dq      5

section .text
global _start
_start:
        mov     rdi, lst
        mov     rsi, qword[len]
        call    sub4
_stop:
        mov     rax, 60
        mov     rdi, 0
        syscall

global sub4
sub4:
        mov     r10, 0
next:
        sub     byte[rdi+r10], 0x30
        inc     r10
        cmp     r10, rsi
        jne     next
        ret
```

(22 points)

Memory	Offset	Value (Hex)	
		before (initial)	after
lst	+4		
lst	+3		
lst	+2		
lst	+1		
lst	+0		

Register	Value (Hex)
	After execution
rsi	

ASCII TABLE

dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char	dec	hex	oct	char
0	0	000	NULL	32	20	040	space	64	40	100	@	96	60	140	`
1	1	001	SOH	33	21	041	!	65	41	101	A	97	61	141	a
2	2	002	STX	34	22	042	"	66	42	102	B	98	62	142	b
3	3	003	ETX	35	23	043	#	67	43	103	C	99	63	143	c
4	4	004	EOT	36	24	044	$	68	44	104	D	100	64	144	d
5	5	005	ENQ	37	25	045	%	69	45	105	E	101	65	145	e
6	6	006	ACK	38	26	046	&	70	46	106	F	102	66	146	f
7	7	007	BEL	39	27	047	'	71	47	107	G	103	67	147	g
8	8	010	BS	40	28	050	(	72	48	110	H	104	68	150	h
9	9	011	TAB	41	29	051	)	73	49	111	I	105	69	151	i
10	a	012	LF	42	2a	052	*	74	4a	112	J	106	6a	152	j
11	b	013	VT	43	2b	053	+	75	4b	113	K	107	6b	153	k
12	c	014	FF	44	2c	054	,	76	4c	114	L	108	6c	154	l
13	d	015	CR	45	2d	055	-	77	4d	115	M	109	6d	155	m
14	e	016	SO	46	2e	056	.	78	4e	116	N	110	6e	156	n
15	f	017	SI	47	2f	057	/	79	4f	117	O	111	6f	157	o
16	10	020	DLE	48	30	060	0	80	50	120	P	112	70	160	p
17	11	021	DC1	49	31	061	1	81	51	121	Q	113	71	161	q
18	12	022	DC2	50	32	062	2	82	52	122	R	114	72	162	r
19	13	023	DC3	51	33	063	3	83	53	123	S	115	73	163	s
20	14	024	DC4	52	34	064	4	84	54	124	T	116	74	164	t
21	15	025	NAK	53	35	065	5	85	55	125	U	117	75	165	u
22	16	026	SYN	54	36	066	6	86	56	126	V	118	76	166	v
23	17	027	ETB	55	37	067	7	87	57	127	W	119	77	167	w
24	18	030	CAN	56	38	070	8	88	58	130	X	120	78	170	x
25	19	031	EM	57	39	071	9	89	59	131	Y	121	79	171	y
26	1a	032	SUB	58	3a	072	:	90	5a	132	Z	122	7a	172	z
27	1b	033	ESC	59	3b	073	;	91	5b	133	[	123	7b	173	{
28	1c	034	FS	60	3c	074	<	92	5c	134	\	124	7c	174	|
29	1d	035	GS	61	3d	075	=	93	5d	135	]	125	7d	175	}
30	1e	036	RS	62	3e	076	>	94	5e	136	^	126	7e	176	~
31	1f	037	US	63	3f	077	?	95	5f	137	_	127	7f	177	DEL

www.alpharithms.com